

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение

высшего образования

«Иркутский государственный университет»

(ФГБОУ ВО «ИГУ»)

Физический факультет

Кафедра теоретической физики

Зав. кафедрой Доцент, к.ф.-м.н.

Ловцов С.В. _____

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

«Преддипломная практика»

Расчёт качественной характеристики уравнения
осцилляций нейтрино в среде по методу Магнуса

Руководитель практики: _____
к.ф.-м.н., доцент, Ломов В.П.

Студент гр. 01411-ДБ
_____/Данеко Илья Игоревич

Работа защищена с оценкой _____
«___» _____ 2026г.

Нормконтролёр
_____/Фролова А.С./

Иркутск 2026

Содержание

Введение	3
1 Уравнение нейтринных осцилляций в среде	4
2 Методы численного интегрирования	7
2.1 Геометрический интегратор	7
2.1.1 Метод М4	9
2.1.2 Обработка данных с метода М4	9
Заключение	15
Список использованной литературы	16
Приложение А	17
Приложение Б	20

Введение

Нейтринная физика сейчас является одним из рубежей современной физики. Также благодаря своим необычным свойствам нейтрино являются важным элементом для построения моделей за пределами стандартной модели.

Нейтрино — это общее название нейтральных фундаментальных частиц с полуцелым спином, участвующих только в слабом и гравитационном взаимодействиях, и относящихся к классу лептонов. Наиболее важным открытием в физике нейтрино стало наблюдение осцилляций, которые к настоящему моменту подтверждены многочисленными экспериментами с солнечными, атмосферными, реакторными и ускорительными нейтрино.

Идея нейтринных осцилляций была предложена Бруно Понтекорво в 1957-1958 годах, она была основана по аналогии с К-мезонами [1, 2]. Уже в 1962 году, Маки, Накагавой и Сакатой была разработана теория смешивания нейтрино в вакууме [3]. В 1963 году Накагавой, Оконики, Сакатой и Тойодой была предложена связь между перемешиванием нейтрино с определенным флейвором и нейтрино с определённой массой [4]. Вольфенштейном было обращено внимание на эффект при распространении нейтрино через обычное вещество (среда электронейтральная, немагнитная, нерелятивистская) в 1978 году [5, 6]. В 1986 году Михеевым и Смирновым этот эффект был физически описан (MSW-эффект) [7, 8]. И наконец, в 2015 году в качестве доказательства осцилляций нейтрино были признаны результаты SNO (Садберийская нейтринная обсерватория) [9] и Super-Kamiokande [10].

По сравнению с вакуумными осцилляциями нейтрино, при изучении осцилляции нейтрино в веществе, все сложнее. Поэтому в таком случае, как правило, применяют численные расчёты.

Основной целью данной работы стоит поиск качественной характеристики численного решения уравнения осцилляции нейтрино в среде.

В первом разделе мы кратко приводим сведения об осцилляциях, в частности осцилляциях нейтрино в веществе. Здесь вводится уравнения осцилляций в веществе.

Во втором разделе мы приводим краткие сведения по альтернативному методу интегрирования уравнения шрёдингеровского типа. Получаем данные численного решения уравнения осцилляций в среде с помощью данного метода.

1. Уравнение нейтринных осцилляций в среде

Когда активные флейворные нейтрино распространяются в веществе, на их эволюционное уравнение влияют эффективные потенциалы из-за когерентного взаимодействия со средой посредством реакций нейтрального и заряженного токов. Таким образом, влияние среды можно описать в терминах потенциалов, в которых распространяются нейтрино, зависящим от состава среды, электрической нейтральности, намагниченности (ориентации спинов), скоростей частиц среды.

Когда три известных состояния флейвора $|\nu_\alpha\rangle (\alpha = e, \mu, \tau)$ являются линейными комбинациями состояний $|\nu_i\rangle$ с массами $m_i (i = 1, 2, 3)$, состояния флейворных нейтрино являются суперпозицией состояний массовых нейтрино

$$|\nu_\alpha\rangle = \sum_i U_{\alpha i}^* |\nu_i\rangle, \quad (1)$$

следовательно, массовые состояния нейтрино являются суперпозицией состояний флейворных нейтрино

$$|\nu_i\rangle = \sum_\alpha U_{\alpha i} |\nu_\alpha\rangle. \quad (2)$$

Коэффициенты $U_{\alpha i}$ являются элементами унитарной матрицы смешивания U , называемой матрицей Понтекорво–Маки–Накагавы–Сакаты¹. Стандартная параметризация матрицы PMNS имеет вид

$$U = O_{23} \Gamma O_{13} \Gamma^\dagger O_{12}, \quad (3)$$

где O_{ij} ортогональные матрицы, представляющие повороты на углы $\theta_{ij} \in [0, \pi/2]$ в соответствующих плоскостях

$$O_{23} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c_{23} & s_{23} \\ 0 & -s_{23} & c_{23} \end{pmatrix}, \quad O_{13} = \begin{pmatrix} c_{13} & 0 & s_{13} \\ 0 & 1 & 0 \\ -s_{13} & 0 & c_{13} \end{pmatrix}, \quad O_{12} = \begin{pmatrix} c_{12} & s_{12} & 0 \\ -s_{12} & c_{12} & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad (4)$$

в то время как $\Gamma = \text{diag}(1, 1, e^{i\delta})$ — диагональная матрица, где $\delta \in [0, 2\pi]$ — фаза CP нарушение, но ниже мы считаем её равной нулю. Здесь $c_{ij} = \cos \theta_{ij}$, а $s_{ij} = \sin \theta_{ij}$, где θ_{ij} углы смешивания.

Явный вид унитарной матрицы смешивания такой:

$$U = \begin{pmatrix} c_{12}c_{13} & s_{12}c_{13} & s_{13}e^{i\delta} \\ -s_{12}c_{23} - c_{12}s_{23}s_{13}e^{i\delta} & c_{12}c_{23} - s_{12}s_{23}s_{13}e^{-i\delta} & c_{13}s_{23} \\ s_{12}s_{23} - c_{12}c_{23}s_{13}e^{i\delta} & -c_{12}s_{23} - s_{12}c_{23}s_{13}e^{i\delta} & c_{13}c_{23} \end{pmatrix}. \quad (5)$$

Рассмотрим нейтрино ν_α рождённое в момент времени t_0 в среде. Состояние системы $|\Psi(t)\rangle$

¹Pontecorvo–Maki–Nakagawa–Sakata (PMNS)

для момента времени $t \geq t_0$ можно представить в виде

$$|\psi(t)\rangle = \sum_{\beta} \psi_{\beta}(t) |\nu_{\beta}\rangle, \quad (6)$$

причём $\psi_{\beta}(t_0) = \delta_{\alpha\beta}$. Вероятность иметь состояние флейвора β в точке в точке $r = t(\hbar = c = 1)$ равна

$$P_{\alpha\beta}(r) = |\psi_{\beta}(r)|^2. \quad (7)$$

После того, как нейтрино покидают среду, амплитуды эволюционируют в соответствии с уравнением, которое управляет вакуумными колебаниями, решение которого проще, если записать его в базисе массовых собственных состояний. Используя уравнение (1) и обозначив амплитуду вероятности нахождения $|\nu_j\rangle$ на краю среды, для $r \geq r_*$ через $A_j = \phi_j(r_*)$, мы получаем

$$\psi_{\beta}(r) = \sum_{j=1}^3 U_{\beta j} A_j e^{-iE_j L} \quad (8)$$

где $E_j = \sqrt{|\mathbf{p}|^2 + m_j^2}$, а $L = r - r_*$ это расстояние пройденное нейтрино в вакууме. Теперь в уравнении (7) мы используем выражение (8) и получаем

$$P_{\alpha\beta} = \sum_j |U_{\beta j}|^2 |A_j|^2 + 2 \sum_{i>j} \text{Re}[U_{\beta i} U_{\beta j}^* A_i A_j^* e^{-i\Delta_{ij} L}]. \quad (9)$$

Величина $\Delta_{ij} = \Delta m_{ij}^2 / 2E$, для $E = |\mathbf{p}|$, представляет собой волновое число колебаний, связанное с квадратом разности масс $\Delta m_{ij}^2 = m_i^2 - m_j^2$.

В результате задача вычисления $P_{\alpha\beta}$ сводится к определению величин A_j , т.е. нахождению амплитуд собственных массовых состояний $\phi_j(r)$ внутри среды, при начальном условии $\phi_j(r_0) = U_{\alpha j}^*$. Для релятивистских нейтрино, распространяющихся в обычной материи, после вычитания глобальной фазы уравнение эволюции для этих амплитуд имеет вид

$$i \frac{d\Phi(\xi)}{d\xi} = [H_0 + v(\xi) U^\dagger V U] \Phi(\xi), \quad (10)$$

где $\Phi^T(r) = (\phi_1(\xi), \phi_2(\xi), \phi_3(\xi))$, U — матрица смешивания, и $V = \text{diag}(1, 0, 0)$, $\xi = \frac{r}{R_0}$, где $R_0 = 6,96 \times 10^5$ — солнечный радиус. Первый член в скобках — это гамильтониан, который управляет эволюцией флейвора в вакууме $H_0 = \text{diag}(0, \Delta_{21}, \Delta_{31})$, он имеет вид:

$$H_0 = \frac{a}{E} \begin{pmatrix} 0 & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (11)$$

Здесь E — числовое значение энергии нейтрино в МэВ, а $a = 4,35196 \times 10^6$ и $b = 0,030554$ — безразмерные параметры. Второй член в скобках учитывает эффекты, связанные с когерентным взаимодействием нейтрино с фоновыми частицами. Функция $v(\xi)$ представляет собой

эффективный потенциал среды.

Так как O_{12} и Γ коммутируют, матрица PMNS может быть записана так: $U = O_{23}\Gamma O\Gamma^\dagger$, где $O = O_{13}O_{12}$. Также коммутаторы $[V, O_{23}]$, $[V, \Gamma]$, $[H_0, \Gamma]$ равны нулю. Следовательно, можно получить

$$i\frac{d\Psi(\xi)}{d\xi} = [H_0 + v(\xi)W]\Psi(\xi), \quad (12)$$

где $\Psi(\xi) = \Gamma^\dagger\Phi(\xi)$, а $W = O^\dagger V O$. То есть

$$W = \begin{pmatrix} c_{13}^2 c_{12}^2 & c_{12} s_{12} c_{13}^2 & c_{12} c_{13} s_{13} \\ c_{12} s_{12} c_{13}^2 & s_{12}^2 c_{13}^2 & s_{12} c_{13} s_{13} \\ c_{12} s_{13} c_{13} & s_{12} c_{13} s_{13} & s_{13}^2 \end{pmatrix},$$

где $c_{ij} = \cos \theta_{ij}$, а $s_{ij} = \sin \theta_{ij}$. Были использованы следующие значения, взятые из [11]

$$\begin{aligned} \sin^2 \theta_{12} &= 0,308, \\ \sin^2 \theta_{23} &= 0,437, \\ \sin^2 \theta_{13} &= 0,0234. \end{aligned} \quad (13)$$

Также необходимо задать определённую форму функции $v(\xi)$. Для Солнца мы используем экспоненциальный профиль электронной плотности

$$v(\xi) = \gamma e^{-\eta\xi}, \quad (14)$$

где $\gamma = 6,5956 \times 10^4$ и $\eta = 10,54$.

Как правило, осциллирующие интерференционные члены в уравнении (9) в среднем равны нулю для нейтрино, пролетающих большое расстояние до Земли. При таких обстоятельствах средняя вероятность обнаружения ν_β в детекторе на Земле определяется некогерентной суперпозицией

$$\langle P_{\alpha\beta} \rangle = \sum_j |U_{\beta j}|^2 \rho_j, \quad (15)$$

где $\rho_j = |A_j|^2 = |\phi_j(r_*)|^2$, решение уравнения (12).

Поскольку

$$\sum_j^3 \rho_j = 1, \quad (16)$$

а $|\psi_j(r_*)|^2 = |\phi_j(r_*)|^2$ мы имеем

$$\sum_j^3 |\psi_j(r_*)|^2 = 1. \quad (17)$$

Если начальное состояние соответствует электронному нейтрино, то

$$\Psi(\xi_0) = \begin{pmatrix} c_{13} c_{12} \\ s_{12} c_{13} \\ s_{13} \end{pmatrix}. \quad (18)$$

2. Методы численного интегрирования

Нам нужно решить уравнение (12). Для решения таких уравнений есть разные методы. Проведём небольшой обзор этих методов.

Большинство задач в физике приводят к дифференциальным уравнениям как первого, так и второго порядков, одной или нескольких переменных. В этой связи перед нами часто встаёт задача найти решение задачи

$$\frac{dy}{dt} = g(t, y), \quad y(t_0) = y_0. \quad (19)$$

Что касается уравнения второго порядка, то его можно представить как систему двух уравнений первого порядка.

Методы решения можно разделить на точные, приближённо-аналитические и численные. В точных методах решение разыскивается через элементарные, специальные функции или в виде интегралов от них. В приближённо-аналитических методах решение ищется либо как предел $y(t)$ некоторой последовательности функций $y_k(t)$, причём $y_k(t)$ выражается через элементарные, специальные функции или интегралы от них, либо как конечный набор таких последовательностей (например, аппроксимация полиномами). Численные — это методы приближенного вычисления решения $y(t)$ на интервале (t_0, T) . К этим методам относят семейство методов Рунге–Кутта и методы геометрического интегрирования.

Ниже мы детально рассмотрим методы Рунге–Кутта, но суть подхода в том, что рассматриваемый интервал разбивают на части $t_1, t_2, \dots, t_N$ с постоянным шагом

$$t_{n+1} = t_n + h, \quad n = 0, 1, 2, \dots \quad (20)$$

Здесь множество точек t_i называется узлами сетки, а h — шага сетки.

2.1 Геометрический интегратор

Обратимся снова к уравнению (12). Его особенности: матричное уравнение, матрица в правой части не коммутирует сама в собой в разных точках и самое важное — она эрмитова.

Для его решения рассмотрим более общую задачу. Пусть дана матрица $A(t)$ $n \times n$ и уравнение вида

$$\frac{dy(t)}{dt} = A(t)y(t), \quad y(t_0) = y_0, \quad A \in C^{n \times n}, \quad y \in C^n. \quad (21)$$

Для решения уравнения (21) воспользуемся подходом, описанным в [11, 12, 13]. Он называется методом Магнуса. Суть метода заключается в представлении искомой функции в виде

$$y(t) = e^{\Omega(t, t_0)} y_0, \quad (22)$$

причём

$$\Omega(t, t_0) \in C^{n \times n}, \quad \Omega(t_0, t_0) = 0. \quad (23)$$

Когда матричные элементы A являются независимыми от t , тогда $\Omega(t, t_0) = (t - t_0)A$. В общем

случае $\Omega(t, t_0)$ задаётся в виде разложения в ряд

$$\Omega(t, t_0) = \sum_{k=1}^{\infty} \Omega_k(t, t_0), \quad \Omega(t_0, t_0) = 0 \quad (24)$$

где слагаемое Ω_k состоит из кратных интегралов от вложенных коммутаторов k матриц A , вычисленных в k разных моментах времени. Сложность отдельных слагаемых в значительной степени возрастает с увеличением индекса k . В частности, первые три из них считаются следующим образом

$$\begin{aligned} \Omega_1 &= \int_{t_0}^t dt_1 A_1, \\ \Omega_2 &= \frac{1}{2} \int_{t_0}^t dt_1 \int_{t_0}^{t_1} dt_2 [A_1, A_2], \\ \Omega_3 &= \frac{1}{6} \int_{t_0}^t dt_1 \int_{t_0}^{t_1} dt_2 \int_{t_0}^{t_2} dt_3 ([A_1, A_2], A_3] + [A_1, [A_2, A_3]]), \end{aligned} \quad (25)$$

здесь $A(t_i) = A_i$, а квадратные скобки обозначают коммутатор: $[A, B] = AB - BA$.

Согласно формулам (22), (25) нам нужно вычислять интегралы. Они при численных вычислениях, тоже считаются числом. Для этого мы можем взять множество из узлов сетки $t_1, t_2, \dots, t_N$, для которых n -ый шаг: $h_n = t_{n+1} - t_n$, где $0 < n < N - 1$.

Далее определим что

$$y(t_{n+1}) = e^{\Omega(t_n + h_n, t_n)} y(t_n). \quad (26)$$

После этого благодаря итерации мы можем записать решение в N шагов

$$y(t_{n+1}) = \prod_{n=0}^{N-1} e^{\Omega(t_n; h_n)} y_0, \quad \Omega(t_n; h_n) = \Omega(t_n + h_n, t_n). \quad (27)$$

Для применения такого метода ряд (24) нужно усечь

$$\Omega^{(p)}(t, t_0) = \sum_{k=1}^p \Omega_k(t, t_0), \quad (28)$$

чтобы понять после какого члена усекаль, необходимо узнать порядок аппроксимации метода. Он определяется, как порядок r -го разложения Тейлора, которое даёт $y(t_n + h_n)$ вплоть до $O(h_n^{r+1})$ из точного значения $y(t_n)$. Поскольку порядок квадратур, необходимый для численной аппроксимации Ω_k в усечённый ряд не превышает порядка аппроксимации r . А так как метод Магнуса симметричен по времени, для достижения метода интегрирования порядка $2r$ (при $r > 1$) в разложении в ряд (24) требуются только члены до $p = 2r - 2$.

Система дифференциальных уравнений, управляющая эволюцией амплитуд трёх нейтрино

в веществе (12), она подходит под тип уравнений, описанный в (21), при $n = 3$, $t = \xi$ и

$$A = -iH, \quad y = \Psi, \quad (29)$$

где $H = H_0 + v(\xi)W$.

Теперь мы применим подход, описанный выше, для численного решения такого уравнения с начальным условием (18). Из (26) получим

$$\Psi(\xi_{n+1}) = e^{\Omega^{[r]}(\xi_n; h_n)} \Psi(\xi_n), \quad (30)$$

где $\Omega^{[r]}$ обозначает приближение к усечённому ряду Магнуса (28) порядка r по h_n

2.1.1 Метод М4

При $p = 2$ в усечённом ряду (28), мы получаем метод порядка $r = 4$. Если соответствующие интегралы Ω_1 и Ω_2 из (25) аппроксимируются двухточечным правилом Гаусса-Лежандра [14]. В частности, учитывая Квадратурные точки (узлы)

$$\xi_{\pm} = \xi_n + (1 \pm \frac{1}{\sqrt{3}}) \frac{h_n}{2}, \quad (31)$$

где $\xi_- < \xi_+$ и

$$H_{\pm} = H(\xi_{\pm}), \quad (32)$$

одноступенчатый показатель интегрирования порядка 4 может быть приведён к виду

$$\Omega^{[4]}(\xi_n; h_n) = -i(H_+ + H_-) \frac{h_n}{2} + \frac{\sqrt{3}}{12} [H_-, H_+] h_n^2. \quad (33)$$

Здесь, как и прежде, квадратные скобки обозначают коммутатор.

Вычисляя уравнение (33) раскрыв матрицу H , мы приходим к

$$\Omega^{[4]}(\xi_n; h_n) = -i(H_0 + \frac{1}{2}(\nu_+ + \nu_-)W)h_n + \frac{\sqrt{3}}{12}(\nu_+ - \nu_-)[H_0, W]h_n^2, \quad (34)$$

где $\nu_{\pm} \equiv \nu(\xi_{\pm})$ — единственные величины, которые необходимо повторно вычислять на каждом шаге интегрирования. Матрица $[H_0, W]$ создаётся только один раз в начале процесса интегрирования.

2.1.2 Обработка данных с метода М4

Перед нами всё также стоит задача найти качественные характеристики численного решения уравнения осцилляции нейтрино в среде. Так как данные полученные с помощью Дормана-Принса в **Wolfram Mathematica** имели выбросы было принято решение сделать такие же расчёты с помощью метода М4 вместо Дормана-Принса. Также мы использовали вместо **Wolfram**

Mathematica код на **Python** для алгоритма и **magexp** программа, для численного решения при помощи метода Магнуса, а именно М4.

Мы поставили задачу найти качественные характеристики численного решения уравнения осцилляции нейтрино в среде. Из численного анализа мы заметили, что реальная и мнимая часть ψ_1 часто пересекают ось абсцисс, однако к правой границе области они выходят на асимптотическое поведение, более не пересекая ось абсцисс. Из подобных соображений было принято решение узнать как распределены нули ψ_1 . Для этого было введено понятие квазипериода. Квазипериод — это длина отрезка содержащего в себе 3 точки в которых функция (в нашем случае реальная или мнимая часть ψ_1) зануляется, причём 2 из них являются границами этого отрезка.

Мы можем считать квазипериод функцией от точки ξ_0 интервала $[0,1; 1]$ в таком смысле: находим 3 ближайших к ξ_0 значения ξ при которых функция зависящая от ξ (в нашем случае реальная или мнимая часть $\psi_1(\xi)$) зануляется. Расстояние между крайними из них и есть искомый квазипериод в точке ξ_0 .

Нас заинтересовало поведение квазипериода на всём интервале $[0,1; 1]$. Вполне логично построить график зависимости квазипериода от ξ . Для получения данных по всему интервалу мы воспользовались тем, что на правом краю интервала ψ_1 выходит на асимптотическое поведение более не пересекая ось абсцисс. Это означает, что есть самый крайний правый интервал с нулями и самый крайний квазипериод. Начиная с него и двигаясь влево мы получили необходимые данные. Для этого мы находим крайний правый квазипериод. Следующим этапом берём интервал последнего вычисленного квазипериода и смещаем его на $\frac{2}{3}$ последнего вычисленного квазипериода влево. Далее ищем все зануления функции зависящей от ξ (в нашем случае реальной и мнимой части $\psi_1(\xi)$) внутри получившегося интервала и ищем квазипериод по приведённому выше алгоритму приравняв середину получившегося интервала к ξ_0 . Повторяем этапы начиная с получения нового интервала до тех пор пока не выполниться одно из двух условий. Либо в новом интервале окажется меньше 3 точек, либо количество вычисленных квазипериодов превысит заранее заданное число (мы выбрали 100). На рис. 1 мы построили график, основываясь на полученных таким образом данных для реальной и мнимой части ψ_1 . $\text{Re } \psi_1$ и $\text{Im } \psi_1$ имеют очень похожий вид. Как видно из графика, размер квазипериода по мере увеличения ξ растёт как будто экспоненциально.

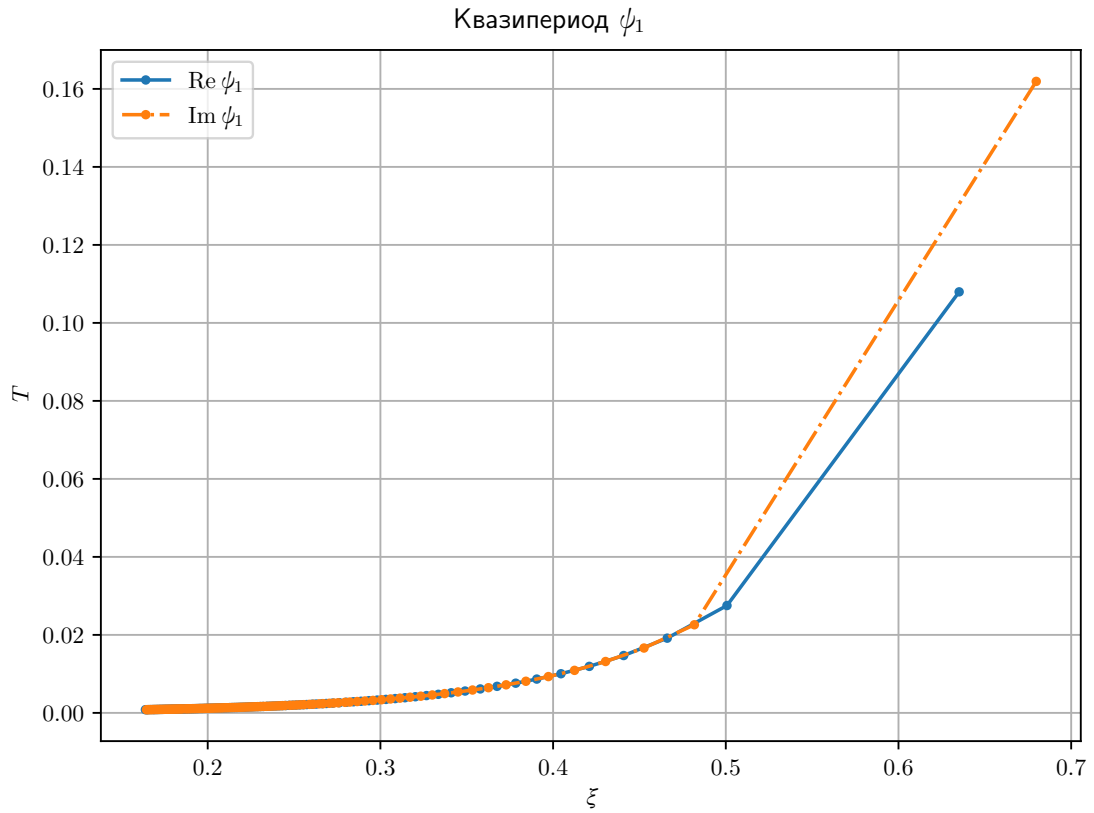


Рисунок 1 — График зависимости длины квазипериода $\text{Re } \psi_1$ и $\text{Im } \psi_1$ от ξ по данным М4

Нам стало интересно насколько чаще реальная и мнимая часть $\psi_{2,3}$ пересекают ось абсцисс по сравнению с реальной и мнимой частью ψ_1 соответственно. Для этого мы посчитали количество занулений реальной и мнимой частей $\psi_{2,3}$ на интервалах ранее посчитанных квазипериодов реальной и мнимой частей ψ_1 соответственно. Основываясь на этих данных, мы построили графики рис. 2, 3. Заметные на графиках выбросы, как мы считаем, являются особенностью работы алгоритма вычисления количество нулей на отрезке.

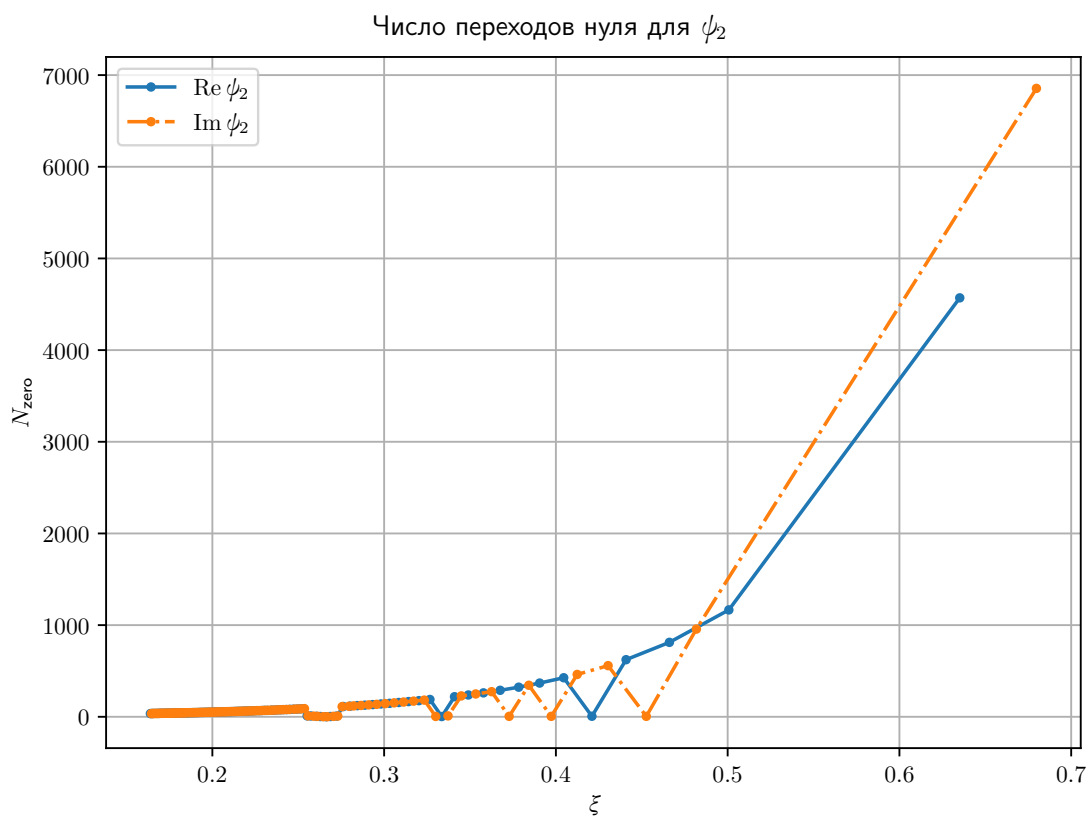


Рисунок 2 — График количества переходов через ноль $\text{Re } \psi_2$ и $\text{Im } \psi_2$ на интервале одного квазипериода ψ_1 по данным М4

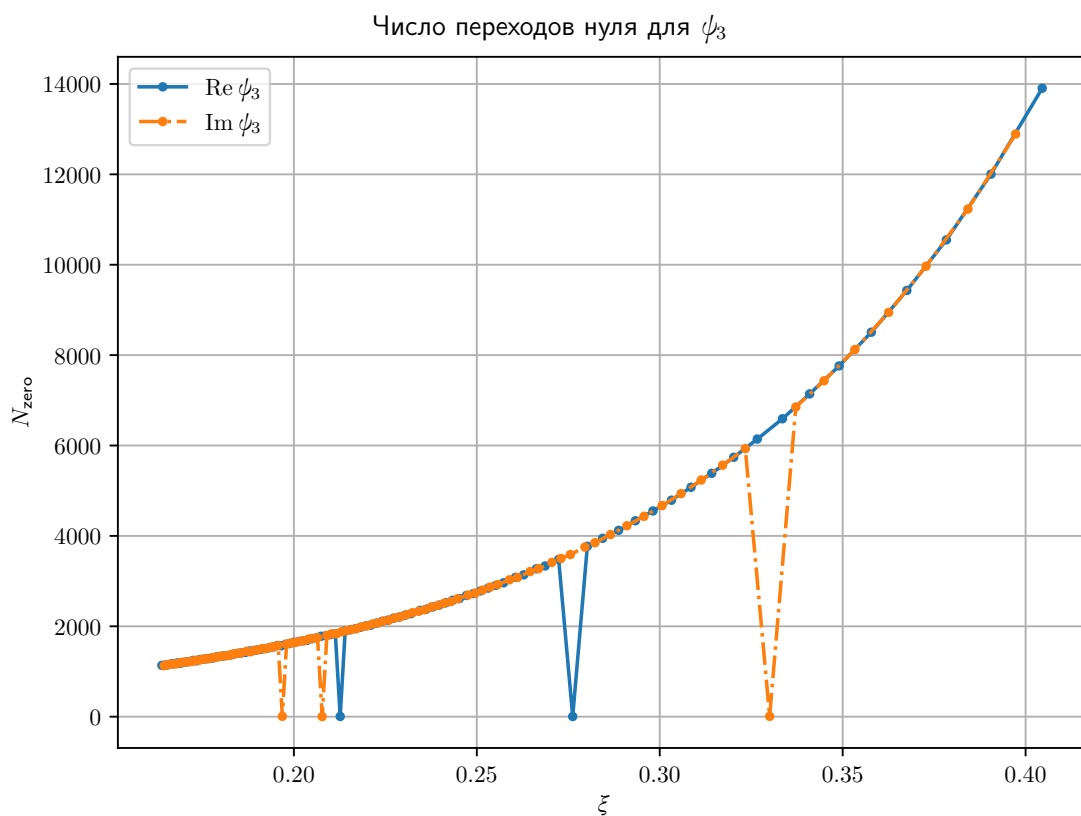


Рисунок 3 — График количества переходов через ноль $\text{Re } \psi_3$ и $\text{Im } \psi_3$ на интервале одного квазипериода ψ_1 по данным М4

Обратим внимание, что на графиках для ψ_1 (рис. 1) мы видим, что величины квазипериодов для реальной и мнимой частей мало отличаются в близких точках. Примерно такое же поведение мы видим в количестве нулей для ψ_2 и ψ_3 (рис. 2, 3).

Схожий характер графиков длин квазипериодов ψ_1 и графиков количества переходов $\psi_{2,3}$ через ноль в квазипериоде ψ_1 позволяет отнормировать данные второй и третьей компоненты по первой. Так, поделив количество нулей $\psi_{2,3}$ в каждом квазипериоде ψ_1 на длину этого квазипериода, мы получим для каждого квазипериода ψ_1 количество нулей $\psi_{2,3}$ на единицу длины, или же величину обратную среднему расстоянию между нулями. На рис. 4, 5 приведены графики основанные на полученных таким образом данных. Их вид очень похож на почти постоянный, если исключить ранее замеченные выбросы. Их почти постоянность можно признать качественным свойством. Их вид подсказывает, что это величина относительно стабильная и может быть использована для качественной характеристики численного решения. Кроме того, для реальной и мнимой частей этот параметр практически один и тот же. Аналогичное поведение мы наблюдаем и для третьей компоненты.

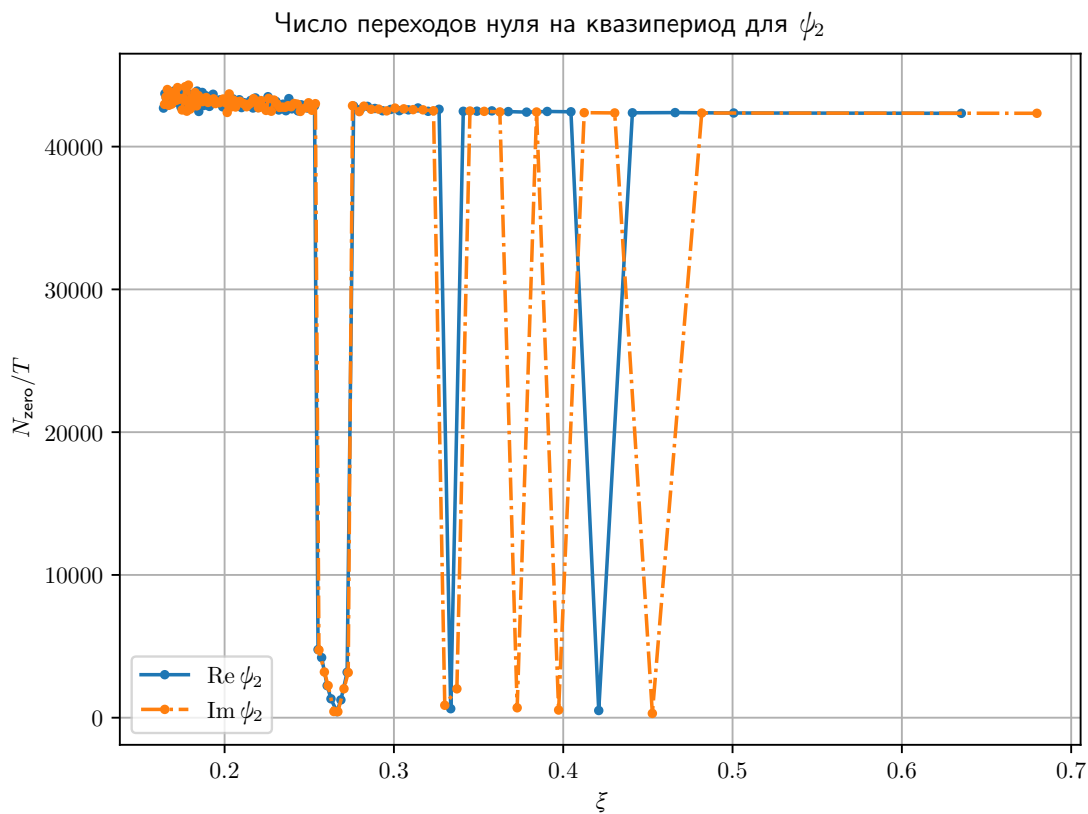


Рисунок 4 — График количество нулей $\text{Re } \psi_2$ и $\text{Im } \psi_2$ на единицу длины квазипериода ψ_1 по данным М4

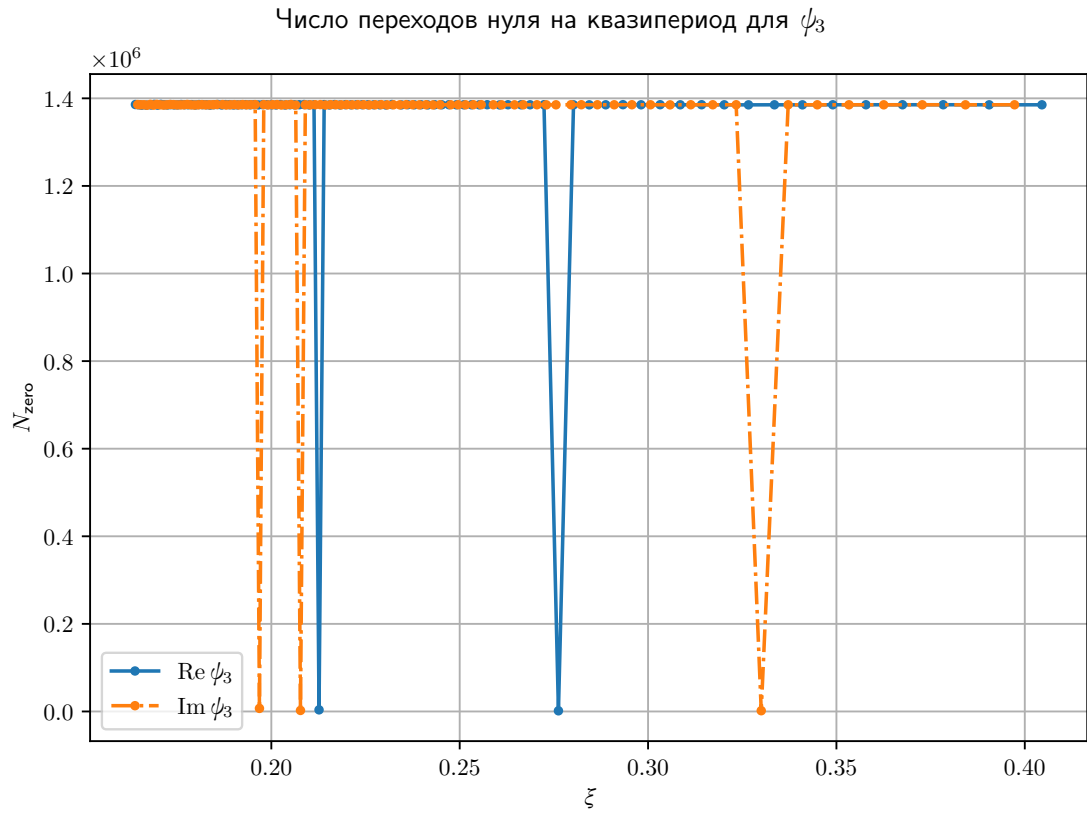


Рисунок 5 — График количество нулей $\text{Re } \psi_3$ и $\text{Im } \psi_3$ на единицу длины квазипериода ψ_1 по данным М4

В прил. Б приведен код с помощью которого были получены данные о квазипериодах $\text{Re } \psi_1$ и $\text{Im } \psi_1$, а также количестве занулений функций $\text{Re } \psi_{2,3}$ и $\text{Im } \psi_{2,3}$ в интервалах ранее посчитанных квазипериодов $\text{Re } \psi_1$ и $\text{Im } \psi_1$. Усечённые графики, не включающие выбросы приведены в прил. А.

Заключение

В работе рассматривались качественные характеристики численного решения уравнения осцилляций нейтрино в среде (12). Применение метода Магнуса позволило выявить, что количество занулений реальной и мнимой части $\psi_{2,3}$ на единицу длины квазипериода реальной и мнимой части ψ_1 является качественной характеристикой численного решения, и она совпадает при использовании разных методов.

Напоследок отметим, что для уменьшения риска человеческой ошибки были разработаны сценарии для автоматизации большинства операций.

Список использованной литературы

- [1] Pontecorvo B. Mesonium and anti-mesonium / B. Pontecorvo // Sov. Phys. JETC. — 1957. — Т. 6. — С. 429.
- [2] Pontecorvo B. Inverse beta processes and nonconservation of lepton charge / B. Pontecorvo // Sov. Phys. JETC. — 1958. — Т. 7. — С. 172–173.
- [3] Maki Z. Remarks on the unified model of elementary particles / Z. Maki, M. Nakagawa, S. Sakata // High-energy physics. Proceedings, 11th International Conference, ICHEP'62, Geneva, Switzerland, Jul 4-11, 1962. — С. 663–666.
- [4] Possible existence of a neutrino with mass and partial conservation of muon charge / M. Nakagawa [et al.] // Prog. Theor. Phys. — 1963. — Т. 30. — С. 727–729.
- [5] Wolfenstein L. Neutrino Oscillations in Matter / L.Wolfenstein // Phys. Rev. — 1978. — Т. D17. — С. 2369–2374.
- [6] Wolfenstein L. Effects of Matter on Neutrino Oscillations / L.Wolfenstein // AIP Conf. Proc. — 1978. — Т. 52. — С. 108–112.
- [7] Mikheev S. P. Neutrino oscillations in matter / S. P. Mikheev, A. Y. Smirnov // 12th International Conference on Neutrino Physics and Astrophysics (Neutrino 86) Sendai, Japan, June 3-8, 1986. — С. 177–193.
- [8] Mikheev S. P. Resonance Oscillations of Neutrinos in Matter / S. P. Mikheev, A. Y. Smirnov // Sov. Phys. Usp. — 1987. — Т. 30. — С. 759–790.
- [9] Measurement of the rate of $\nu_e + d \rightarrow p + p + e^-$ interactions produced by 8B solar neutrinos at the Sudbury Neutrino Observatory / Q. R. Ahmad [et al.] // Phys. Rev. Lett. — 2001. — Т. 87. — С. 071301. —
- [10] Evidence for oscillation of atmospheric neutrinos / Y. Fukuda [et al.] // Phys. Rev. Lett. — 1998. — Т. 81. — С. 1562–1567.
- [11] Casas F. Efficient numerical integration of neutrino oscillations in matter / F. Casas, J.C. D'Olivo, J.A.Oteo // Phys. Rev. D. — 2016. — Т. 94, no. 11. — С. 113008.
- [12] Шайдурова А. В. Использование разложения Магнуса для численного решения уравнений осцилляций нейтрино в среде [Текст] : bathesis : 03.03.02 — Физика / Шайдурова Арина Владимировна; ИГУ. — Иркутск, 2019. — 52 с.
- [13] A pedagogical approach to the magnus expansion / S. Blanes [et al.] // European Journal of Physics. — 2010. — Т.31 — С.907–918.
- [14] ALie group methods / A. Iserles [et al.] // Acta Numerica. — 2000. — Т.9 — С.215.

Графики по данным М4, без выбросов

График количество нулей $\text{Re } \psi_{2,3}$ и $\text{Im } \psi_{2,3}$ на единицу длины квазипериода ψ_1 по данным М4 очень похож на почти постоянный, если исключить выбросы. Но из-за этих самых выбросов масштаб оси $\frac{N_{\text{zero}}}{T}$ большой. Поэтому рассмотрим такие графики не включающие выбросы.

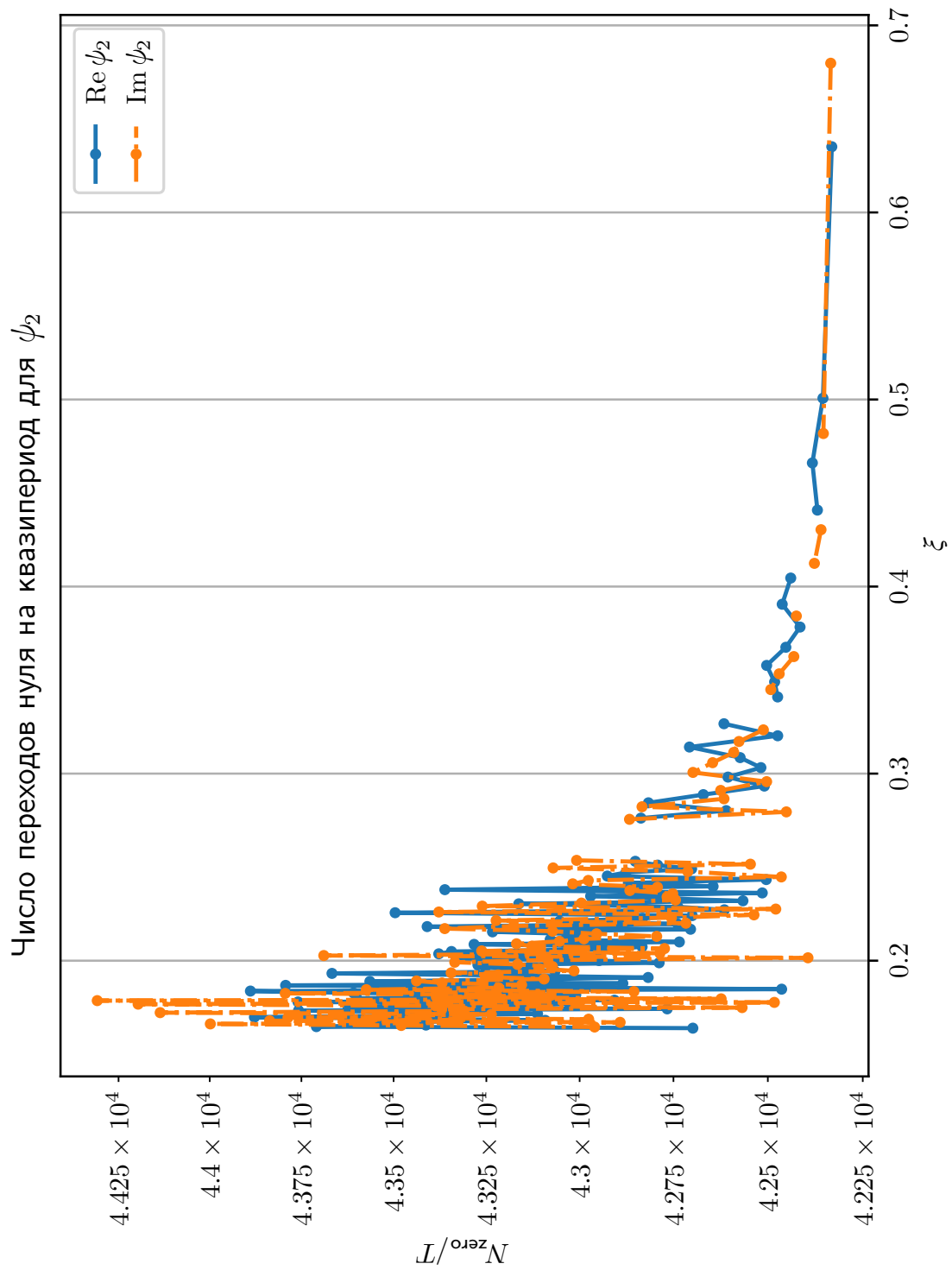


Рисунок 6 — График количество нулей $\text{Re } \psi_2$ и $\text{Im } \psi_2$ на единицу длины квазипериода ψ_1 по данным M4, без выбросов

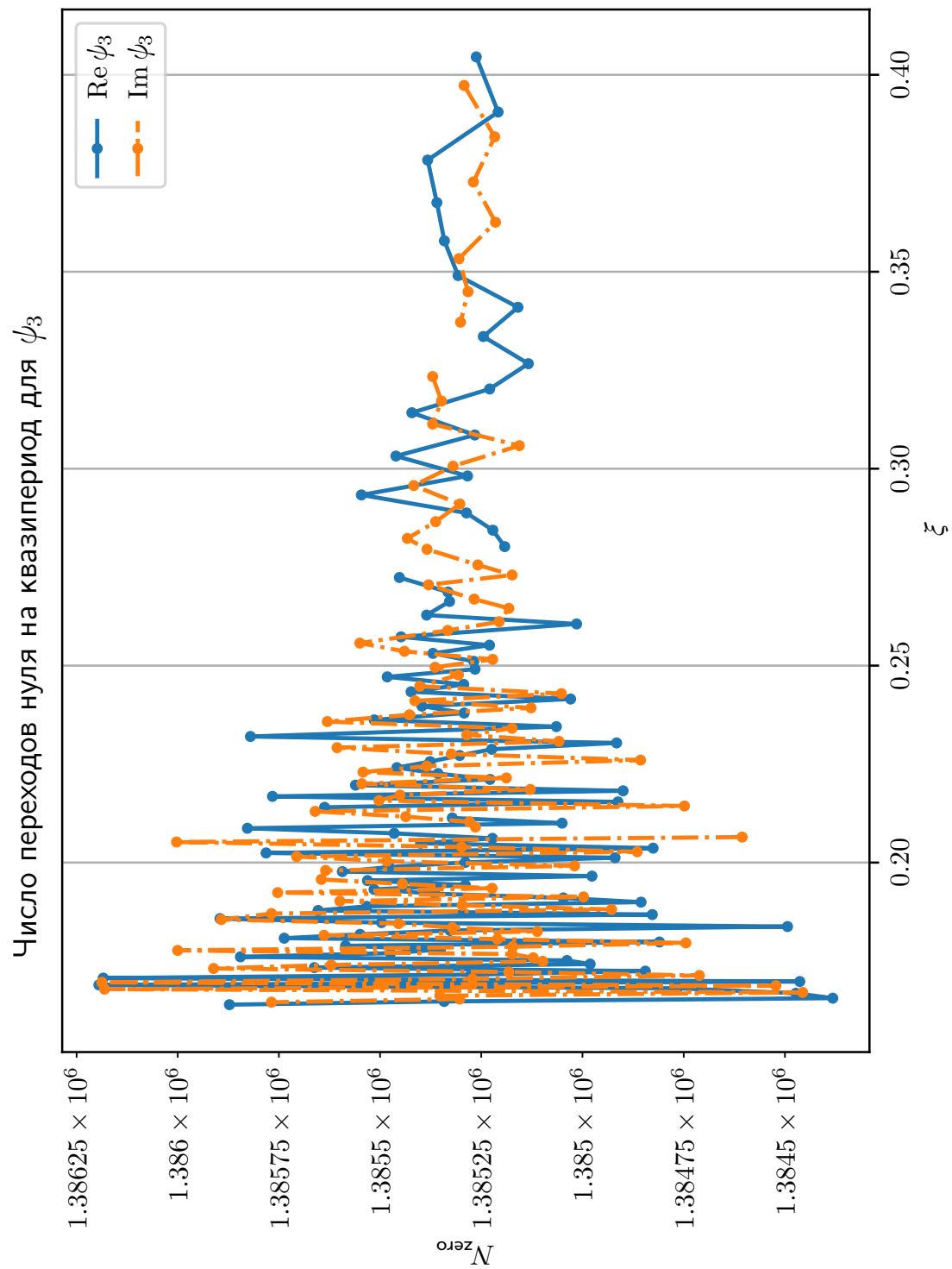


Рисунок 7 — График количество нулей $\text{Re } \psi_3$ и $\text{Im } \psi_3$ на единицу длины квазипериода ψ_1 по данным M4, без выбросов

Код в Python для решения уравнения осцилляции нейтрино в среде

Для численного решения уравнения (12) использовался код на python.

Ниже представлен код сценария использованного для получения данных по квазипериодам $\text{Re } \psi_1(\xi)$, $\text{Im } \psi_1(\xi)$ в зависимости от ξ , а также данных по количеству переходов через ось абсцисс $\text{Re } \psi_{2,3}$ и $\text{Im } \psi_{2,3}$ на интервале каждого из ранее найденных квазипериодов $\text{Re } \psi_1$ и $\text{Im } \psi_1$ соответственно.

Его также можно найти в репозитории <https://github.com/Fest-Fut/Kursovoy>.

```

1  #!/usr/bin/python
2
3  import sys
4  sys.path.append("./magexp/playground/python")
5  import os
6
7  # If there is no magexp directory (actually, the above path), this will fail!
8  import magexp4 as m4
9
10 import numpy as np
11 from pathlib import Path
12 import datetime
13
14
15 SMDIR = "magexp"
16
17
18 def _step1(fh, mode):
19     """
20     Run first step: calculate quasi-period intervals for the first component.
21     """
22
23     match mode:
24         case 're':
25             qprR1 = funcQPeriodStat(eRpsi1, 10, 100)
26
27             fname = fh("qperiod_eRpsi1.dat")
28             np.savetxt(fname, qprR1)
29
30             fname = fh("qperiod_eRpsi1.npy")
31             np.save(fname, qprR1)
32
33         case 'im':
34             qprI1 = funcQPeriodStat(eIpsi1, 10, 100)
35
36             fname = fh("qperiod_eIpsi1.dat")
37             np.savetxt(fname, qprI1)
38
39             fname = fh("qperiod_eIpsi1.npy")
40             np.save(fname, qprI1)
41
42         case _:
43             print(f"[ERROR:_step1] unsupported mode: '{mode}'")
44
45
46 def _step2(fh, mode, idx):

```

```

47 """
48 Run routine to calculate then number of zeroes for the second component in an
49 interval.
50
51 The data file with array of intervals must be located under 'OUTPUT' dir and
52 has name 'qperiod_eR/Ipsi1.npy' (only binary format is supported).
53 """
54
55 match mode:
56     case "re":
57         dat0 = fh("qperiod_eRpsi1.npy")
58         _title = "RE-PSI2"
59         _fh = eRpsi2
60         odat = fh(f"{idx}kolNull_eRpsi2.dat")
61
62     case "im":
63         dat0 = fh("qperiod_eIpsi1.npy")
64         _title = "IM-PSI2"
65         _fh = eIpsi2
66         odat = fh(f"{idx}kolNull_eIpsi2.dat")
67
68     case _:
69         print(f"[ERROR:_step2] unsupported mode: '{mode}'")
70         return
71
72 with open(dat0, 'rb') as f:
73     _dat = np.load(f)
74     _ldt = len(_dat)
75     shft = 0
76
77     if Path(odat).exists():
78         _dt = np.loadtxt(odat)
79         if _dt.ndim == 1:
80             _idx = int(_dt[0])
81         else:
82             _idx = int(_dt[-1][0])
83         if _idx % 2 != idx:
84             print(f"[ERROR] the last entry in '{odatt}' is wrong for mode '{idx}'")
85             sys.exit(1)
86         if _idx + 2 > _ldt:
87             print(f"[WARNING] everything seems already be calculated: last item {_idx}, total number of elements {_ldt}")
88             return
89         data = _dat[_idx+2::2]
90         shft = _idx + 2
91     else:
92         data = _dat[idx::2]
93         shft = idx
94
95     funMasivNullInRangesQPeriod(_fh, data, 10, _title, odat, shft)
96
97
98 def _step3(fh, mode, idx):
99     """
100 Run routine to calculate the number of zeroes for the third component in an
101 interval.
102
103 The data file with array of intervals must be located under 'OUTPUT' dir and
104 has name 'qperiod_eR/Ipsi1.npy' (only binary format is supported).
105 """
106
107 match mode:
108     case "re":
109         dat0 = fh("qperiod_eRpsi1.npy")

```

```

110     _title = "RE-PSI3"
111     _fh = eRpsi3
112     odat = fh(f"{idx}kolNull_eRpsi3.dat")
113
114     case "im":
115         dat0 = fh("qperiod_eIpsi1.npy")
116         _title = "IM-PSI3"
117         _fh = eIpsi3
118         odat = fh(f"{idx}kolNull_eIpsi3.dat")
119
120     case _:
121         print(f"[ERROR:_step3] unsupported mode: '{mode}'")
122         return
123
124     with open(dat0, 'rb') as f:
125         _dat = np.load(f)
126         _ldt = len(_dat)
127         shft = 0
128
129         if Path(odat).exists():
130             _dt = np.loadtxt(odat)
131             if _dt.ndim == 1:
132                 _idx = int(_dt[0])
133             else:
134                 _idx = int(_dt[-1][0])
135             if _idx % 2 != idx:
136                 print(f"[ERROR] the last entry in '{odat}' is wrong for mode '{idx}'")
137                 sys.exit(1)
138             if _idx + 2 > _ldt:
139                 print(f"[WARNING] everything seems already be calculated: last item {_idx}, total number of elements {_ldt}")
140                 return
141             data = _dat[_idx+2::2]
142             shft = _idx + 2
143         else:
144             data = _dat[idx::2]
145             shft = idx
146
147     funMasivNullInRangesQPeriod(_fh, data, 10, _title, odat, shft)
148
149
150 def main(args):
151     """
152     'Main' function.
153     """
154
155     if len(args) != 3:
156         print("[ERROR] script expects two parameters: mode and 'index' (odd/even).")
157         sys.exit(1)
158
159     os.chdir(SMDIR)
160     if not m4.prepareBinDir():
161         sys.exit(f"Failed to prepare bin directory of '{SMDIR}', see message above.")
162
163     os.chdir("./bin")
164
165     outdir = "../..data/magexp"
166
167     def filn(name):
168         return f"{outdir}/{name}"
169
170     mode = args[1]
171     idx = int(args[2]) % 2
172

```

```

173     match mode:
174         case 'psi1':
175             _step1(filn, "re")
176             _step1(filn, "im")
177         case 're-psi1':
178             _step1(filn, 're')
179         case 'im-psi1':
180             _step1(filn, 'im')
181         case 'psi2':
182             _step2(filn, "re", idx)
183             _step2(filn, "im", idx)
184         case 're-psi2':
185             _step2(filn, 're', idx)
186         case 'im-psi2':
187             _step2(filn, 'im', idx)
188         case 'psi3':
189             _step3(filn, "re", idx)
190             _step3(filn, "im", idx)
191         case 're-psi3':
192             _step3(filn, 're', idx)
193         case 'im-psi3':
194             _step3(filn, 'im', idx)
195         case _:
196             print(f"[ERROR] unrecognized mode: '{mode}'")
197
198
199 def eRpsi1(x):
200     """
201     Wrapper for real part of first component
202     """
203
204     prob, _, _, rpsi1, ipsi1, rpsi2, ipsi2, rpsi3, ipsi3, err = m4.sol(x)
205     return float(rpsi1)
206
207
208 def eIpsi1(x):
209     """
210     Wrapper for imaginary part of first component
211     """
212
213     prob, _, _, rpsi1, ipsi1, rpsi2, ipsi2, rpsi3, ipsi3, err = m4.sol(x)
214     return float(ipsi1)
215
216
217 def eRpsi2(x):
218     """
219     Wrapper for real part of second component
220     """
221
222     prob, _, _, rpsi1, ipsi1, rpsi2, ipsi2, rpsi3, ipsi3, err = m4.sol(x)
223     return float(rpsi2)
224
225
226 def eIpsi2(x):
227     """
228     Wrapper for imaginary part of second component
229     """
230
231     prob, _, _, rpsi1, ipsi1, rpsi2, ipsi2, rpsi3, ipsi3, err = m4.sol(x)
232     return float(ipsi2)
233
234
235 def eRpsi3(x):

```

```

236 """
237 Wrapper for real part of third component
238 """
239
240 prob, _, _, rpsi1, ipsi1, rpsi2, ipsi2, rpsi3, ipsi3, err = m4.sol(x)
241 return float(rpsi3)
242
243
244 def eIpsi3(x):
245     """
246     Wrapper for imaginary part of third component
247     """
248
249     prob, _, _, rpsi1, ipsi1, rpsi2, ipsi2, rpsi3, ipsi3, err = m4.sol(x)
250     return float(ipsi3)
251
252
253 def funcNullRanges(fn, x1, x2, n):
254     """
255     Определяем с заданной «точностью» подынтервалы, где зануляется функция.
256
257     Возвращаем массив подынтервалов.
258
259     Функция проводит «грубую» оценку числа «нулевых» интервалов (где зануляется
260 функция).
261     """
262
263     t = []
264     xx = np.linspace(x1, x2, n)
265     mm = [fn(x) for x in xx]
266     for i in range(len(xx)-1):
267         # print(f"[INFO:funcNullRanges] {i=} {mm[i]=}")
268         if mm[i]*mm[i+1] < 0:
269             t.append([xx[i], xx[i+1]])
270
271     return t
272
273
274 def funcNullRangesStab(fn, x1, x2, n):
275     """
276     Определяем до заданной предельной точности подынтервалы, где есть только один
277 нуль.
278
279     Возвращаем границы интервала.
280     """
281
282     tt = []
283     imx = 6
284     n1 = n2 = -1
285     _name = "INFO:funcNullRangesStab"
286
287     for i in range(imx):
288         nx = n*(4 + i)**i + 1
289         print(f"[{_name}] {nx=}")
290         tt = funcNullRanges(fn, x1, x2, nx)
291         n1 = n2
292         n2 = len(tt)
293         if n1 == n2:
294             break
295
296     return tt
297
298

```

```

299 def funcTochnNull(fn, x1, x2, eps):
300     """
301     Определяем с указанной точностью точное положение нуля.
302
303     Возвращаем границы интервала.
304     """
305
306     t1 = x1
307     t2 = x2
308     while (t2-t1) > eps:
309         if fn(t1)*fn((t1+t2)/2) < 0:
310             t2 = (t1+t2)/2
311         else:
312             t1 = (t1+t2)/2
313
314     return [t1,t2]
315
316
317 def funcQPRangeEnvelop(fn, x0, x1, x2):
318     """
319     Ищет ближайший ноль к x0 в диапазоне [x1,x2] и выдает предыдущий и последующие нули
320     """
321
322     i0 = 0
323     tt = []
324     x1 = []
325     # xc = []
326     xr = []
327     eps = 1e-8
328     _name = "ERROR:funcQPRangeEnvelop"
329     _nm = "DEBUG:funcQPRangeEnvelop"
330
331     tt = np.array(funcNullRangesStab(fn, x1, x2, 10))
332     ttmin = np.min(np.abs(tt[:,0] - x0))
333     i0 = np.where( np.abs(tt[:,0] - x0) == ttmin)[0][0]
334     if i0 > len(tt) - 1:
335         print(f"[{_name}] the smallest is the last one!")
336         return True
337     """
338     print(f"[{_nm}] {i0=}, {tt=}, tt[:,0] - x0 = {tt[:,0] - x0}")
339     """
340     x1 = tt[i0 - 1]
341     # xc = tt[i0]
342     xr = tt[i0 + 1]
343     x1 = funcTochnNull(fn, x1[0], x1[1], eps)
344     xr = funcTochnNull(fn, xr[0], xr[1], eps)
345
346     return [x1[0], xr[1]]
347
348
349 def funcLastQPeriod(fn, n):
350     """
351     Найдти положение крайнего правого нуля и вернуть интервал его содержащий.
352
353     Пояснение:
354
355     Из предварительного анализа известно, что на правом конце есть последний
356     интервал с нулём, который не доходит до правой границы. Задача функции -
357     определить положение этого нуля и его «изолирующий» интервал.
358     """
359
360     x0 = 0.50
361     x2 = 0.8

```

```

362     tt = []
363     y1 = y2 = y3 = 0
364     eps = 1e-8
365     _name = "INFO:funcLastQPeriod"
366
367     tt = funcNullRangesStab(fn, x0, x2, n)
368     k = len(tt)-1
369     print(f"[{_name}] seredina funcLastQPeriod")
370     y1 = funcTochnNull(fn, tt[k-2][0], tt[k-2][1], eps)
371     y2 = funcTochnNull(fn, tt[k-1][0], tt[k-1][1], eps)
372     y3 = funcTochnNull(fn, tt[k][0], tt[k][1], eps)
373
374     return [y1, y2, y3]
375
376
377 def funcQPeriodStat(fn, nSeed, nSteps):
378     """
379     Обязательная документация к функции.
380     """
381
382     tabQPeriod = []
383     endt = []
384     x00 = 0
385     x0 = 0.11
386     gran1 = []
387     dx = 0
388     t = 0
389     _name = "INFO:funcQPeriodStat"
390     _nm = "WARNING:funcQPeriodStat"
391     _nnm = "DEBUG:funcQPeriodStat"
392
393     endt = funcLastQPeriod(fn, nSeed)
394     print(f"[{_name}] Konec funcLastQPeriod")
395     dx = endt[2][0] - endt[0][0]
396     tabQPeriod.append([endt[0][0], endt[2][0]])
397     for i in range(nSteps+1):
398         t = tabQPeriod[i][0]
399         if t - 3.*dx/2 < x0:
400             print(f"[{_nm}] reached the lower bound '{x0}' on step '{i}'")
401             return tabQPeriod
402         x00 = t - 3*dx/4
403
404         """
405         print(f"[{_nnm}] {i=}, t - 1.5dx = {t-3/2*dx}, {x00=}, {t=}, {dx=}")
406         """
407         print(f"[{_name}] {i=}")
408         gran1 = funcQPRangeEnvelop(fn, x00, x00-dx, x00+dx)
409         dx = np.abs(gran1[1] - gran1[0])
410         tabQPeriod.append(gran1)
411
412     return tabQPeriod
413
414
415 def funMasivNullInRangesQPeriod(fn, qpr, n, name, odat, shft):
416     """
417     Подсчитывает количество переходов через ноль данной функции на заданных
418     интервалах.
419
420     Результаты сразу записываем в файл.
421     """
422
423     _nam = f"funMasivNullInRagesQPeriod: '{name}'"
424

```

```

425     for i in range(len(qpr)):
426         t = datetime.datetime.now()
427         print(f"[{_nam}]: {t} -- {i}/{len(qpr)} -- ({qpr[i][0]}, {qpr[i][1]})")
428         _num = len(funcNullRangesStab(fn, qpr[i][0], qpr[i][1], n))
429         with open(odat, "a") as f:
430             print(f"{2*i+shft:d}\t{qpr[i][0]:.18e}\t{qpr[i][1]:.18e}\t{_num:d}", file=f, flush = True)
431
432     return
433
434
435 if __name__ == "__main__":
436     try:
437         main(sys.argv)
438     except KeyboardInterrupt:
439         sys.exit("Interrupted by user")

```